Part 1 — Frontend Team (Person 1)

> “My part is the frontend. I use React to create the login page and separate dashboards for Admin, HOD, and Teacher. I

keep reusable buttons, forms, and menus in components. React sends requests to the backend to read or save information.”

1. Introduction to Frontend

The Frontend is the part of the website that users can see and use.

For example, when we open the College Portal, we see the login page, buttons, menus, forms, tables, and dashboards. All these things are part of the Frontend.

We are using React.js to build the Frontend.

React helps us create different pages and reuse the same parts, such as buttons, menus, and forms, on multiple pages.

2. Frontend Pages Creation and Their Purpose

Our portal will have different pages for different users.

Page	Purpose
Login Page	Allows users to sign in
Admin Dashboard	Allows the Admin to manage college information
HOD Dashboard	Shows information for the HOD's department
Teacher Dashboard	Shows the teacher's own permitted information
Department Page	Shows and manages departments
Teacher Page	Shows teacher profiles
Notices Page	Displays college announcements
Timetable Page	Shows timetable information
No Access Page	Tells users when they are not allowed to open a page

Each user should see the features they are allowed to use.

For example, an Admin can manage departments, while a Teacher may only need to view their own timetable.

3. Frontend Folder Structure and it's Organization

```
frontend/
└── src/
    ├── pages/
    │   ├── Login.jsx
    │   ├── AdminDashboard.jsx
    │   ├── HODDashboard.jsx
    │   └── TeacherDashboard.jsx
    │
    ├── components/
    │   ├── Button.jsx
    │   ├── Navbar.jsx
    │   └── Form.jsx
    │
    ├── features/
    │   ├── departments/
```

```
│   ├── teachers/
│   ├── announcements/
│   └── timetable/
│
├── services/
│   └── apiClient.js
│
├── App.jsx
└── main.jsx
```

What do these folders mean?

- pages: Contains complete website pages, such as the Login page and dashboards.
- components: Contains small parts that can be reused, such as buttons and menus.
- features: Keeps related code together, such as everything related to teachers.
- services: Contains the code that sends requests to the Backend.
- App.jsx: Connects the main pages and navigation.
- main.jsx: Starts the React application.

We keep these folders separate so the code is easy to find, understand, and update.

4. Navigation Between Pages

We will use React Router to navigate between pages without needing to reload the entire website every time.

The process will be:

```
User opens website
        |
v
     Login Page
        |
v
   Login successful?
        |
v
  Check user's role
        |
   +----+-----+
   |    |     |
 Admin  HOD  Teacher
   |    |     |
v    v     v
 Admin  HOD  Teacher
Dashboard Dashboard Dashboard
```

After login, the website will open the dashboard that matches the user's role.

5. Frontend and Backend Communication

The Frontend cannot directly save information into MongoDB. It must send a request to the Backend.

For example, when the Admin adds a teacher:

1. **The Admin fills in the teacher form.**
2. **React collects the form information.**
3. **React sends the information to the Backend using an API.**
4. **The Frontend waits for the Backend's reply.**
5. **If the teacher is saved successfully, React shows a success messageand updates the page.**

The Frontend will also show loading messages and useful error messages when something goes wrong.

6. Tools used by the Frontend team

- React.js: Creates website pages.
- Vite: Helps us start and build the React application.
- React Router: Moves users between pages.
- Fetch or Axios: Sends requests to the Backend.
- CSS: Controls the appearance of the website.
- VS Code: Used to write and edit code.
- Git and GitHub: Used to save and share code with the team.

7. Frontend Organization and Code Management

We will use meaningful file names, reusable components, and a consistent design.

We will test forms and navigation and make sure that users see clear messages when data is loading, when a page is empty, or when an error occurs.

Frontend person's conclusion and handover

The Frontend is responsible for what users see and click. React creates the pages, displays information, and sends requests through APIs.

However, the Frontend cannot decide on its own whether a request is allowed or save information directly into the database. It needs another part to check the request and do the actual work. That next part is the Backend, which the second team member will explain.

Person 2 — Backend

COLLEGE PORTAL / APIs • Authentication • Permissions • MongoDB

> My part is the backend. I use Node.js and Express.js to create APIs. The backend checks login, validates information, checks permissions, and communicates with MongoDB. It does not allow users to access information they are not permitted to see.

1. Introduction to Backend

As the first team member explained, the Frontend shows the website and sends requests when users click buttons or submit forms.

Now, the Backend receives those requests and decides what should happen next.

We are using Node.js and Express.js to build the Backend.

Think of the Backend as a helper between the website and the database.

- Node.js: Allows JavaScript to run on the server.
- Express.js: Helps us create APIs and handle requests.
- API: A way for the Frontend and Backend to communicate.

For example, when an Admin adds a teacher, the Backend checks the request and permissions before asking the database to save the information.

2. Backend Folder Structure and it's Organization

```
backend/
|-- src/
|   |-- routes/
|   |   |-- authRoutes.js
|   |   |-- departmentRoutes.js
|   |   `-- teacherRoutes.js
|   |
|   |-- controllers/
|   |   |-- authController.js
|   |   |-- departmentController.js
|   |   `-- teacherController.js
|   |
|   |-- models/
|   |   |-- User.js
|   |   |-- Department.js
|   |   `-- Teacher.js
|   |
|   |-- middleware/
|   |   |-- auth.js
|   |   `-- checkRole.js
|   |
|   |-- config/
|   |   `-- database.js
|   |
|   `-- app.js
|
|-- server.js
`-- package.json
```

What do these folders mean?

- **routes**: Contains the API addresses that receive requests.
- **controllers**: Handles requests and prepares responses.
- **models**: Describes how information should be stored in MongoDB.
- **middleware**: Checks login, permissions, and other rules.
- **config**: Contains settings, including the database connection.
- **app.js**: Sets up Express and connects the routes and middleware.
- **server.js**: Starts the Backend server.
- **package.json**: Contains project information and required packages.

We separate these folders so that each file has a clear job. This makes the Backend easier to understand, test, and fix.

3. APIs and Their Usage

An API is like a messenger between the Frontend and Backend.

The Frontend asks for something, and the Backend processes the request and sends back a reply.

For example:

API request	What it does
POST /api/v1/auth/login	Checks login details
POST /api/v1/auth/logout	Logs the user out
GET /api/v1/auth/me	Returns information about the logged-in user
GET /api/v1/departments	Gets the department list
POST /api/v1/departments	Creates a department
GET /api/v1/teachers	Gets the permitted teacher list
GET /api/v1/announcements	Gets announcements
GET /api/v1/timetables	Gets timetable information

The two common API methods are:

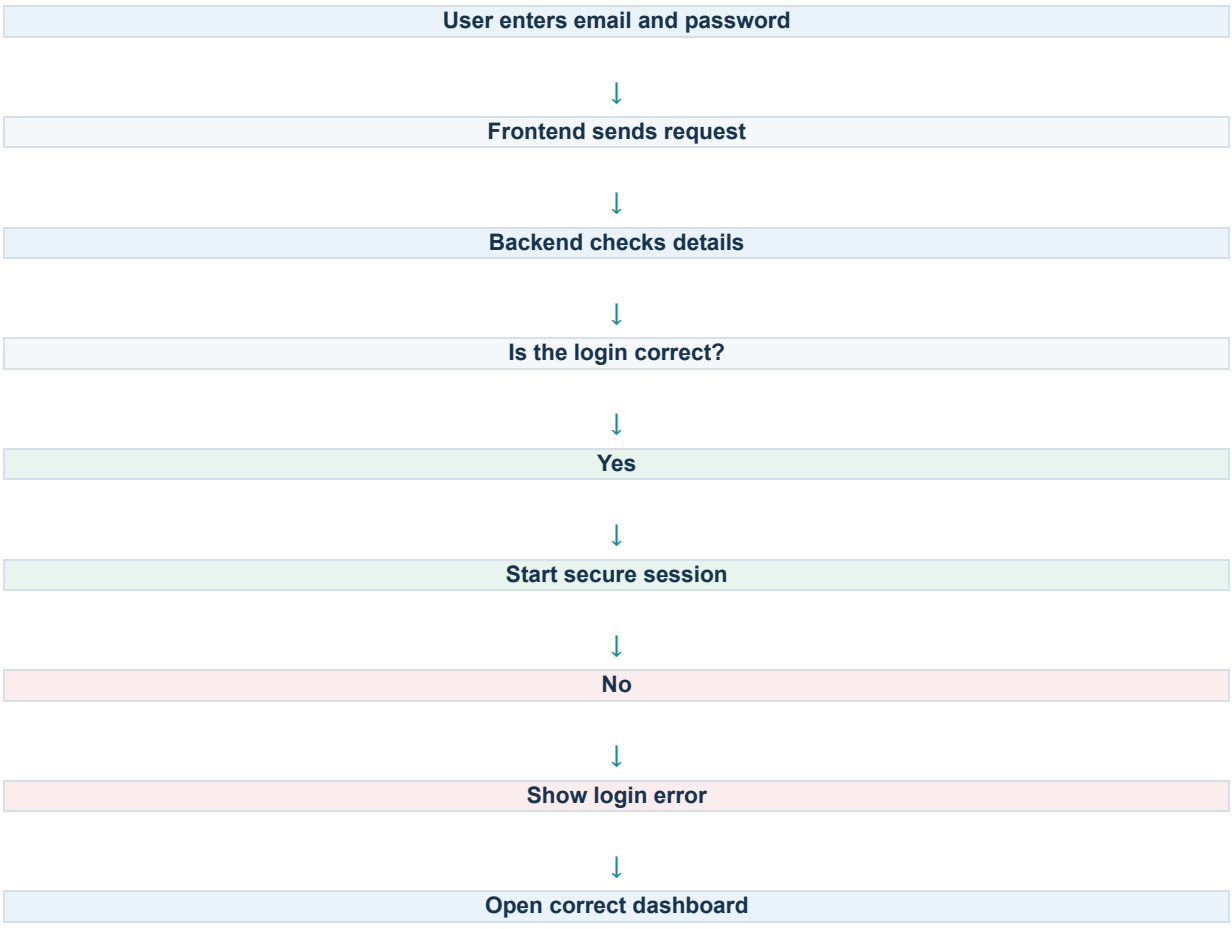
- GET: Used to read information.
- POST: Commonly used to send information or create a new record.

We will also use methods such as PATCH to update records and DELETE when deletion is permitted.

4. Login and Authentication Process

Suppose a teacher enters their email and password.

The process is:



The Backend will:

1. Check whether the required login information has been entered.
2. Find the user's account.
3. Compare the entered password with its securely stored password hash.
4. Start a secure session if the login is correct.
5. Return the user's permitted role information.
6. Reject the request if the login is incorrect.

5. Roles and Permissions Management

Our portal has three main roles.

Role	What the Backend allows
Admin	Manage college departments and user accounts according to permissions
HOD	Manage permitted information for their assigned department
Teacher	Access their own permitted profile and timetable information

For example, if an HOD from the Computer Science department tries to access restricted information from another department, the Backend must reject the request.

This check must happen on the server every time protected information is requested. We cannot depend only on the Frontend to hide restricted pages.

6. Backend and MongoDB Communication

The Backend uses a database library such as Mongoose to communicate with MongoDB.

For example, when the Frontend sends a request to add a teacher:

1. The Backend receives the request.
2. It checks the user's login and permissions.
3. It checks whether the submitted information is valid.
4. It asks Mongoose to save the teacher's information in MongoDB.
5. MongoDB saves the record and returns a result.
6. The Backend sends a success or error response to the Frontend.

The Backend is the only application layer that should connect directly to MongoDB.

7. Tools Used by the Backend Team

- Node.js: Runs JavaScript code on the server.
- Express.js: Creates APIs and handles requests from the frontend.
- Mongoose: Connects the backend to MongoDB and helps validate data.
- MongoDB: Stores college portal data, such as users, teachers, departments, and announcements.

Backend person's conclusion and handover!

The Backend receives requests from the Frontend, checks the user's login and permissions, and decides whether an action is allowed. It uses APIs to communicate with the Frontend and Mongoose to communicate with MongoDB.

But the Backend needs a proper place to store the college's information. It also needs to know how departments, teachers, users, and timetables are related. This is the responsibility of the third team member, who will explain Database & Integration.

Person 3 — Database & Integration

College Portal • MongoDB, integration, setup, and security

> “My part is the database and integration which will be done by using MongoDB. MongoDB stores users, departments, teachers, notices, and timetables in collections. The backend connects to MongoDB, and I help test that the frontend, backend, and database work together correctly.

1. Introduction to Database

As the second team member explained, the Backend checks requests and asks the database to save or find information.

The Database stores the information so that it can be used again later.

We are using MongoDB for our College Portal.

MongoDB stores information in collections. Think of a collection as a cupboard containing one type of information.

For example, one collection contains users, another contains departments, and another contains teachers.

2. MongoDB Collections and Their Purpose/ Need

Collection	What it stores
users	Login details, roles, and password hashes
departments	Department names, codes, and assigned HODs
teachers	Teacher profiles and department references
announcements	Notices, authors, and intended audience
timetables	Teachers, subjects/classes, rooms, days, and times
academicRecords	Approved academic information when that module is developed
documents	Document details and access information
auditLogs	Important actions, such as role changes

We will begin with the collections needed for the first version. More collections can be added when new modules are developed.

3. Relationships Between MongoDB Collections

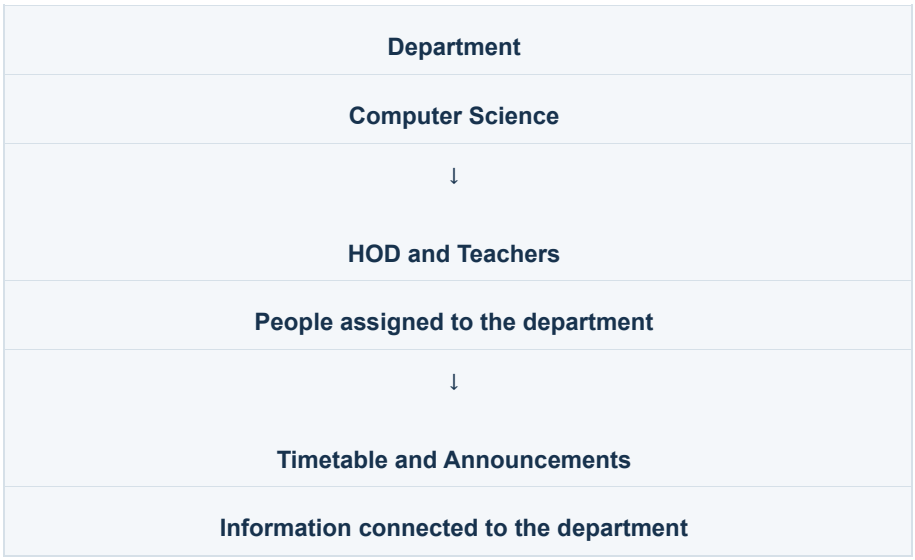
The records are related to each other.

For example:

- One department can have many teachers.
- A department can have an assigned HOD.

- Every user account has a role, such as Admin, HOD, or Teacher.
- An announcement has an author.
- A timetable entry refers to the relevant teacher and department or class.

Here is a simple diagram:



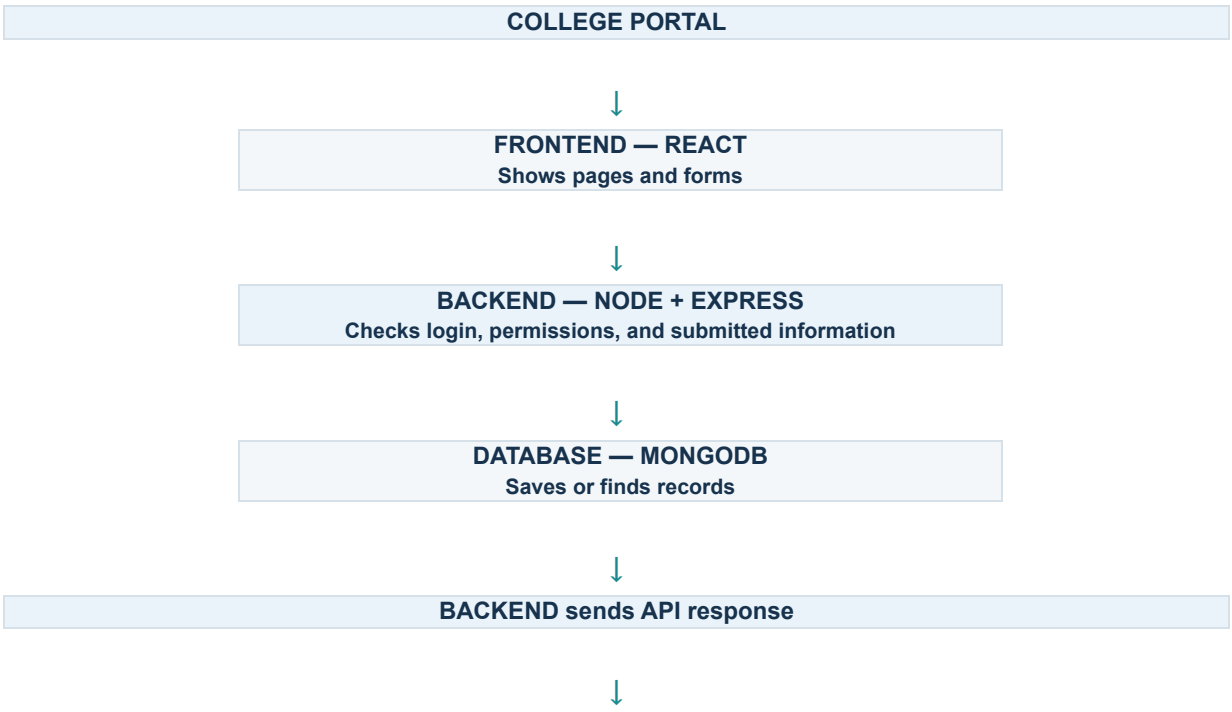
This is a simple relationship diagram. In MongoDB, related records can refer to one another using IDs.

For example, a teacher record can contain a department ID. This helps the system know which department the teacher belongs to.

We will also add unique rules for information such as email addresses and department codes so that accidental duplicates can be avoided.

4. Integration of Frontend, Backend, and Database

This is the most important part of the project.



FRONTEND shows the result

Let's understand this with one example.

Example: Admin adds a new teacher

Step 1 — Frontend

The Admin fills in the teacher's name, email, and department, then clicks Save.

↓

Step 2 — Backend

The Backend checks whether the Admin is logged in, has permission, and entered valid information.

↓

Step 3 — Database

MongoDB stores the new teacher's record if the request passes the checks.

↓

Step 4 — Result

The Backend sends a response to React, and React displays a success message and the updated teacher list.

This is how the Frontend, Backend, and Database work together as one application.

5. Installation and Setup Instructions

Step 1: Install the tools

Install Node.js LTS, Git, and Visual Studio Code. Use either MongoDB Atlas or a local MongoDB installation.

Postman can be used to test APIs.

Step 2: Create the React frontend

Open a terminal in the location where the project should be created and run:

```
npm create vite@latest frontend -- --template
react cd frontend npm install npm run dev
```

Vite will show a local development address, usually http://localhost:5173.

Step 3: Set up the backend

Open a second terminal in the main college-portal folder and run:

```
mkdir backend cd backend npm init -y
npm install express mongoose dotenv
cors npm install --save-dev nodemon
```

Create the backend folders and files shown in Section 1 as the team implements each module.

Step 4: Configure environment variables

Create backend/.env for local development. Example values:

```
PORT=5000
MONGODB_URI=your_mongodb_connection_string
```

```
FRONTEND_URL=http://localhost:5173
```

Use the actual connection string from the selected MongoDB setup. Do not commit .env or real passwords/API keys to GitHub. Keep only placeholder examples in .env.example.

Step 5: Connect and run

The backend should connect to MongoDB using Mongoose, start the Express server, and expose API routes. The frontend calls those routes using the API service. Add suitable npm scripts to backend/package.json after creating server.js.

Step 6: Test the setup

- Confirm the React page opens in the browser.
- Confirm the backend server starts without errors.
- Confirm the database connection succeeds.
- Use Postman to test API endpoints as they are implemented.

6. Database Security and all Data Protection

College information should only be available to the right people.

We will follow these rules:

- Keep database passwords and secret keys out of GitHub.
- Allow the Backend, not the React application, to connect to MongoDB.
- Use a database account with only the permissions it needs.
- Allow users to access only the records permitted by their roles.
- Make sure an HOD cannot access another department's restricted records.
- Validate information before saving it.
- Use fake personal data while developing and testing.
- Keep backups and test whether they can be restored.
- Protect uploaded documents and check who can download them.
- Record important actions, such as changes to user roles, without recording passwords or secret tokens.

7. Integration and Testing Process

Integration means connecting the three separate parts and checking that they work together.

Our steps will be:

1. The Frontend person creates the pages.
2. The Backend person creates the APIs.
3. The Database person creates the collections and connection setup.
4. We agree on the same field names and API addresses.
5. We connect React to Express and Express to MongoDB.
6. We test login, dashboards, teacher creation, notices, and timetables.

7. We test whether users are blocked from information they are not allowed to access.
8. We fix errors and update the project instructions.

All three people will use Git and GitHub to share changes and keep the project organized.

Database & Integration person's conclusion

My responsibility is to organize how college information is stored in MongoDB and make sure the Backend can save and retrieve it correctly. I will also help connect and test all three parts.

The Frontend shows the information, the Backend checks and processes it, and the Database stores it. When all three work together correctly, our College Portal becomes a working website.